PRODUCT REQUIREMENTS / PILOT EDITION

DirectPG & Society Living

Product, technology and premium design brief

A zero-brokerage accommodation experience built around clear discovery, consent-based onboarding and accountable property operations.

DECISION DOCUMENT | MVP PILOT | INDIA-FIRST ASSUMPTIONS

Document guide

This brief converts the supplied MVP PRD into an implementation-ready product direction. It keeps the single-property pilot narrow while adding the technical decisions, security and governance controls, delivery standards, and a more distinctive premium visual system.

Section	What it answers
Product strategy	Who it serves, which problem the pilot solves, and how success is measured
Product scope	MVP requirements plus prioritized post-pilot additions
Technical plan	Architecture, data boundaries, APIs, delivery and operational controls
Trust and governance	Privacy, consent, security, accessibility, legal review and accountable operations
Premium design direction	Brand feel, interaction principles and page-by-page experience guidance
Delivery checklist	Standards to follow before release and pilot expansion

Working assumption

The first deployment serves one PG or housing society in India. Applicable law, contract enforceability, identity-document handling, payment rules and local property requirements need review by qualified counsel before launch.

Contents

01 Executive direction / 02 Personas and pilot outcomes / 03 Product scope and feature roadmap / 04 User journeys / 05 Technical architecture / 06 Data model and API boundaries / 07 Security and governance / 08 Premium design system / 09 Delivery plan and release gates

01 Executive direction

Product promise. Find and manage a place to live directly, with clear terms and a visible path from inquiry to move-in and maintenance resolution. The pilot should prove reliable resident onboarding and issue handling before investing in an open marketplace.

Pilot boundary

- One partner property, with owner-managed inventory and tenant access.
- Web landing page for property discovery, visit requests and owner interest.
- Mobile app for tenant onboarding, document review, agreement consent and maintenance.
- Owner console for inventory, visit leads, resident records, agreements and tickets.
- Use offline payment recording during the pilot unless payment collection is a proven requirement.

Personas

Persona	Primary job	Pilot outcome
Owner / property admin	Keep occupancy, visits, resident records and maintenance organized	Less follow-up across chat and paper; clear ticket ownership and status
Prospective tenant	Assess a property and arrange a visit with transparent terms	Understand rent, deposit, rules and availability before committing
Resident tenant	Complete move-in requirements and get repairs addressed	Know what documents are held and when an issue will be handled
Staff / technician (optional role)	Receive assigned work and report progress	Only see the jobs and contact details needed to complete work

Pilot success measures

Measure	Target from source PRD	Definition to instrument
Onboarding completion	>90% within 15 minutes	Started onboarding to completed consent; exclude time waiting on owner action
App-based maintenance	>85% of requests	Tickets created in app / all reported maintenance requests during pilot
Resolution time	Establish baseline	Median and 90th percentile from open to resolved, by category and urgency
Web conversion	Establish baseline	Unique property-page visitors who submit a visit request or open a store link
Consent quality	No target supplied	Completion, abandonment and correction rates; investigate disputes or support contacts

02 Product scope and feature roadmap

The first release should close the core loop for one property. Features below are ordered by their contribution to learning and operational value.

MVP: required for pilot

- Property page: verified photos, room and bed availability, rent, deposit, included amenities, rules and contact details. Show full known charges before a visit or reservation request.
- Visit request: name, phone, preferred time and consent to be contacted; owner sees and updates request status.
- Inventory and resident setup: property, rooms, beds, price, deposit, notice period, occupancy and allocation history. Prevent double allocation with a server-side transaction.
- Phone OTP account access and role-based authorization; owners invite residents to the correct property.
- Document vault: collect only required KYC fields and files, with purpose, retention and access explained before upload.
- Agreement workflow: generate a draft, allow both parties to review, capture affirmative consent and OTP verification, then preserve the exact version and event history.
- Maintenance tickets: category, urgency, description, photo, status, owner note, assignment and tenant confirmation or reopen.
- Owner dashboard: upcoming visits, pending onboarding, occupancy and overdue or emergency tickets.

Recommended additions

Priority	Feature	Why it helps / boundary
P1 - pilot	Move-in checklist and document expiry reminders	Makes onboarding completion visible; reminders must be configurable and non-coercive.
P1 - pilot	Ticket SLA labels and update timeline	Set property-specific response expectations; do not promise guaranteed repair times.
P1 - pilot	Availability and price change history	Reduces disputes about the terms shown when a visit or request was made.
P1 - pilot	Owner/resident notification preferences	Allow essential operational alerts and separate optional marketing consent.
P2 - after pilot	Rent ledger and payment receipts	Add only after reconciliation, refunds and failed-payment support are designed.
P2 - after pilot	Verified reviews and move-out/deposit checklist	Collect only from genuine residents; provide a fair correction/dispute path.
P2 - after pilot	Multi-property owner analytics	Scale once the single-property workflow and access boundaries are proven.
Later	Marketplace search, recommendations, chat and service vendors	Requires supply density, moderation, abuse handling and additional privacy controls.

Keep out of the pilot

Avoid microservices, complex recommendation systems, automated KYC decisions, mandatory in-app payment and broad multi-property search until the pilot shows a clear need.

03 User journeys and acceptance criteria

Resident onboarding

- Discover the property and inspect current terms, rules, location and verified images.
- Request a visit or start a resident invitation. State clearly whether a request is non-binding.
- Sign in with phone OTP; confirm property invitation and role.
- Review why each identity field is needed and how long each document is retained. Upload supported file types through a private upload flow.
- Review the rendered agreement on a phone-sized screen; download or request correction before consent.
- Record separate affirmative consent and OTP verification for each party. Provide each party a copy and a way to report an error.

Maintenance resolution

- Resident selects a category and urgency; emergency guidance includes the property’s actual contact path.
- System creates a ticket and sends an acknowledgment with ticket reference.
- Owner assigns or updates status with an ETA note. Each update is time-stamped and visible to the resident.
- Resident confirms resolution or reopens with a reason. Reopened tickets return to the active queue.

Important acceptance rules

- A resident can read only their own profile, agreements, tickets and property information shared with them.
- An owner can access only properties they own or have been granted an explicit staff role on.
- Agreement consent records are append-only; corrections create a new version and preserve the prior version.
- A ticket cannot be closed by an owner without resident confirmation, except through a documented admin override with reason.
- Availability, price, consent and ticket status changes must be attributable to an authenticated actor.

04 Technical architecture

Recommended MVP architecture

Layer	Recommendation	Implementation notes
Web	Next.js + TypeScript	Static/SSR property marketing pages; accessible forms and server-rendered SEO content. Keep owner console behind authentication.
Mobile	React Native + Expo + TypeScript	One iOS/Android app; use platform secure storage for tokens and camera/document upload.
Backend	Supabase (Postgres, Auth, Storage) with server-side functions	Start as a modular monolith. Use server-controlled functions for agreement creation, privileged changes, notification dispatch and audit events.
Database	PostgreSQL	Relational constraints and transactions for inventory, assignments and consent workflow. Enable RLS on all user data tables.
Files	Private Supabase Storage or S3	Short-lived signed access, server MIME/magic-byte validation, size limits, malware scanning where available; never public KYC URLs.
Notifications	Expo Notifications / FCM; SMS OTP provider	Keep message content minimal. Notifications link to authenticated app screens, not documents or sensitive details.
Hosting / delivery	Managed hosting; GitHub Actions CI/CD	Separate development, staging and production. Use managed secrets; backups and restore test before pilot.

System boundaries

Clients call authenticated backend APIs. The backend validates role and property ownership for every object request, then uses database constraints and RLS as a second boundary. Sensitive agreement and document actions run server-side. A background job handles notification retries and retention tasks. A single repository and deployable backend keep the pilot simple; split components only when measured load or team ownership justifies it.

API and reliability rules

- Use resource-oriented REST endpoints and correct HTTP semantics: GET reads, POST creates, PATCH changes selected fields, DELETE only where deletion is allowed. Return consistent 4xx/5xx errors and 409 for conflicts such as double-booking.
- Validate request schemas at the API boundary, paginate lists, cap upload sizes and rate-limit OTP, visit and ticket endpoints.
- Use idempotency keys for retries that create bookings, agreements or payment events.
- Wrap bed allocation and status transitions in database transactions. Add indexes for property_id, tenant_id, status, created_at and common queue filters; verify query plans as data grows.
- Avoid Redis and Kubernetes for MVP. Add caching only after measuring a real bottleneck; introduce container orchestration only when deployment or scaling needs require it.

05 Data model and API boundaries

Core entities

Entity	Key fields and constraints
users / memberships	id, name, phone, role/status; membership links a user to property and permission set. Avoid relying on a single global role.
properties	id, owner_id, name, address, rules, amenities, status; immutable owner audit trail for material edits.
rooms / beds	room_id, property_id, bed_label, rent, deposit, availability; unique bed per property and allocation state protected transactionally.
visits / booking_requests	property_id, requester_id, preferred_at, status, terms_snapshot, consent_to_contact_at. Store request history.
documents	owner_id, property_id, storage_key, document_type, retention_until, status; private object key, never a reusable public URL.
agreements / versions	property_id, bed_id, owner_id, tenant_id, version, rendered_hash, status, consent events, created_at. Preserve rendered copy and event chain.
issues / issue_events	property_id, room_id, tenant_id, category, urgency, status; event table records actor, prior/new status, note and timestamp.
audit_events	actor_id, action, object_type/id, timestamp, request correlation id, minimal security context. Never store OTP values or raw credentials.

Schema and integrity guidance

- Normalize user, property, room, bed and event records. Use foreign keys, check constraints, unique constraints and explicit status enums or validated values.
- Store timestamps in UTC and display in the property's local time zone. Store money as integer minor units plus currency, never floating point.
- Keep current ticket status for fast queues while retaining a full event log. Use soft deletion or tombstones where needed to honor retention and audit obligations.
- Do not store full government ID numbers unless counsel and the operating model establish a necessary purpose. Mask displayed identifiers and restrict access to the smallest staff set.

Example API surface

Area	Illustrative endpoints
Property	GET /v1/properties/{id}; PATCH /v1/owner/properties/{id}; GET /v1/properties/{id}/availability
Visits	POST /v1/visit-requests; GET /v1/owner/visit-requests; PATCH /v1/owner/visit-requests/{id}
Agreements	POST /v1/agreements/drafts; GET /v1/agreements/{id}; POST /v1/agreements/{id}/consents
Tickets	POST /v1/issues; GET /v1/issues; PATCH /v1/issues/{id}/status; POST /v1/issues/{id}/reopen

06 Security, privacy and governance

Build privacy and access control into the first schema and screen design. Controls below are release requirements, not optional polish. The study deck’s legal checklist is a prompt for review; it does not establish which law applies or prove compliance.

Security baseline

- Authentication verifies identity; authorization checks role, property membership and object ownership on the server for every request. Test for broken object-level authorization.
- Enable Supabase RLS on every sensitive table. Policies should be deny-by-default and tested for owner, tenant, staff and unauthenticated users.
- Use parameterized SQL, framework output escaping and a strict Content Security Policy. Protect state-changing browser requests against CSRF and configure explicit CORS origins.
- Validate uploads by size, extension, MIME type and file signature. Store objects in private buckets with no execution permissions; issue short-lived signed URLs after authorization.
- Rate-limit OTP sends and verification attempts, login, uploads and public forms. Add abuse monitoring without logging sensitive payloads.
- Keep secrets server-side, rotate keys, require MFA for privileged owner/admin access and use secure, platform-appropriate token storage. Never rely on a client-side admin flag.
- Verify webhook signatures if payment or messaging webhooks are added. Prevent SSRF if the product later fetches user-supplied URLs.
- Scrub PII, tokens, agreement contents and document identifiers from logs and analytics. Patch dependencies routinely and review production build artifacts for exposed secrets.

Privacy and legal review before launch

- Publish accurate privacy, terms, contact, cancellation/refund and cookie notices as applicable. State data purposes, recipients, retention, deletion path and support contact in plain language.
- Use unchecked, separate choices for optional marketing. Do not bundle marketing consent with required service consent. Avoid dark patterns and disclose all known charges before a booking or payment commitment.
- Set a retention schedule for KYC media, agreement records, ticket photos and backups. Provide a self-serve request or support path for account deletion and access/correction requests, subject to legally required retention.
- Confirm jurisdiction-specific rules for rental/PG agreements, electronic consent, OTP evidence, identity documents, minors, consumer claims, taxes and payment handling with local counsel. Do not market agreements as legally guaranteed.
- Use genuine, permissioned property images and resident reviews. Verify commercial licenses for fonts, photography and icons. Provide a public business identity and support route.

Accessibility and inclusive use

- Target WCAG 2.1 AA: 4.5:1 contrast for normal text and 3:1 for large text; support screen readers, scalable text, keyboard access on web, focus visibility and reduced motion.
- Label form fields, explain errors next to the field, allow OTP resend with clear limits, and avoid communicating status by color alone. Provide alt text for meaningful images.
- Do not accept users under the locally permitted age without a reviewed age and guardian-consent process. Avoid collecting age data unless required.

07 Premium design direction

Design intent

Make the service feel calm, dependable and locally grounded. Premium comes from well-edited content, photography, typography, clear pricing and careful interaction details. Avoid generic gradients, decorative badges, repeated three-card feature grids, emoji headings, heavy blur, arbitrary icon sets and motion that slows the page.

Visual system

Element	Direction
Palette	Deep ink/navy for structure; warm off-white surfaces; restrained teal for actions and status; muted brass as a small highlight. Confirm contrast for every state.
Typography	Pair a distinctive licensed editorial serif for large marketing headlines with a highly legible sans serif for UI and data. Use a compact type scale and generous line-height; test Indian names and numerals.
Photography	Use authentic, permissioned photos of the actual pilot property. Favor daylight, room details and clear captions. Never imply a photo is verified if it is not.
Layout	Use an asymmetric split hero: direct value proposition and actions on the left, property photography or a real product screen on the right. Keep the mobile sequence linear.
Components	Create custom spacing, forms, tables, status labels and buttons. Use icons only when they improve recognition; pair with text labels. Avoid untouched component-library defaults.
Motion	Use short, purposeful transitions only for state feedback. Respect reduced-motion settings; avoid cursor trails, parallax and scroll-triggered spectacle.

Landing page composition

- Hero: "Find your next place directly." Supporting line explains no brokerage on the pilot listing, digital onboarding and repair tracking. Use only claims the operator can substantiate. Primary action: view rooms or schedule a visit. Secondary: owner interest.
- Show the property before generic feature marketing: large verified image, neighborhood, price range, availability date, rules and a transparent fee summary.
- Visit form: concise, accessible and privacy-forward. Confirm the request is not a reservation unless it truly reserves a bed.
- Explain the three moments of the service with real screens: explore, move in, get repairs handled. Feature walkthrough should not use fake metrics or simulated "live" activity.
- End with owner onboarding and real business contact details. Keep download links and QR code only when store distribution is ready.

App experience

- Tenant home: next action and current status first; room details, agreement, documents and issues are easy to find.
- Owner home: pending visits and onboarding first, then occupancy and the active ticket queue with clear urgency and age.
- Agreement review: readable text, version and date visible; separate review and consent steps; no pre-ticked box; provide download and correction path.
- Maintenance: large text status and next update, with urgency instructions that reflect the property’s real response process.

08 Delivery plan and release gates

A four-stage sequence keeps early work focused on the operational loop. Dates should be set after confirming staffing, legal review and the partner property.

Stage	Scope	Exit evidence
1. Validate	Confirm pilot owner, inventory, visit process, agreement template, document purpose/retention and emergency handling. Map current operations.	Signed-off workflow; approved property content; counsel review plan; named support owner.
2. Foundation	Set up environments, auth, property/room model, RLS policies, audit events, web property page and owner console shell.	Access tests pass for each role; backup/restore and deploy path documented.
3. Resident loop	Build invitation/onboarding, private documents, agreement version/consent workflow, ticket lifecycle and notifications.	End-to-end flows work on representative iOS/Android devices; support can find and resolve common issues.
4. Pilot and learn	Train owner/staff; onboard residents; measure conversion, completion, ticket usage and resolution. Fix high-severity usability or security issues.	Pilot review against KPIs; prioritized backlog and explicit decision on expanding beyond one property.

Engineering workflow to follow

- Plan before coding: turn each feature into a short specification, acceptance criteria, edge cases and reviewable tasks.
- Check current official documentation for framework, SDK and provider APIs before implementation; avoid copying stale examples.
- Use Git branches and small, reviewable changes. Require CI to run type checks, linting, unit tests and integration tests for database and API boundaries.
- Review every change for authorization gaps, race conditions, privacy leaks, failure handling and maintainability before merge. Then simplify duplicated or overbuilt code.
- Keep a modular monolith for the pilot. Document the reason before introducing Redis, queues, microservices, Kubernetes or other operational dependencies.

Release checklist

Gate	Required evidence
Product	Price/availability snapshot, clear visit vs booking semantics, complete owner and resident flows, realistic empty/error states.
Security	Role and object access tests, RLS review, upload validation, rate limits, secrets scan, dependency review, log redaction.
Privacy / legal	Approved notices, purpose and retention for each data field, deletion/contact route, reviewed agreement template and consent wording.
Accessibility	Keyboard and screen reader pass on web; contrast, labels, errors, text scaling, reduced motion and mobile touch target checks.
Operations	Owner training, support route, incident contact, backups, restoration evidence, notification fallback and maintenance emergency path.
Measurement	Events defined without sensitive payloads; dashboard/report for funnel and ticket timing; baseline and owner for each KPI.

09 Open decisions before implementation

The following decisions affect the data model or launch readiness and should be resolved during pilot validation.

- Which property and city will host the pilot, and who handles support outside normal hours?
- Does a visit request ever hold a bed, and if so for how long and on what terms?
- Which exact identity fields and files are necessary, who may view them, and when are they deleted?
- What agreement template and electronic consent process will local counsel approve? What happens when a party disputes a record?
- Will tenants use personal phones and numbers, or does the operator need a staff-assisted onboarding path?
- What is the resident communication channel for urgent maintenance when the app or network is unavailable?
- Are payments in scope for the pilot? If yes, specify refund, failure, reconciliation and payment-provider requirements before implementation.
- What accessibility languages are required for the pilot community?

Decision rule

When a decision adds a new category of sensitive data, a payment flow, an external integration or a legal claim, document its purpose, owner, failure path and review before building it.

Source note

Prepared from the user-provided DirectPG & Society Living MVP PRD and the user-provided Tech and Governance Master Study Deck. Deck material was treated as reference guidance and translated into applicable design, engineering, security and governance practices. Legal and regulatory statements in that deck are not treated as validated legal advice.